

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

A Learned Adaptive Inference System for Real-Time YOLOv8 Object Detection on Edge Devices with Pothole Detection as an Application

VISHAL K. BHAT¹, VAIBHAV P. RODAPPANAVAR¹, NAREN SRIDHARAN², HARDIK PANERU², and RAJESWARA RAO K V S³

¹Department of Computer Science, RV College of Engineering, Bengaluru, Karnataka 560059, India (e-mail: vishalkbhat.cs23@rvce.edu.in; vaibhavp.cs23@rvce.edu.in)

²Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, Karnataka 560059, India (e-mail: narensridharan.ec23@rvce.edu.in; hardikpaneru.ec23@rvce.edu.in)

³Department of Industrial Engineering & Management, RV College of Engineering, Bengaluru, Karnataka 560059, India (e-mail: rajeswararaokvs@rvce.edu.in)

Corresponding author: Rajeswara Rao K V S (e-mail: rajeswararaokvs@rvce.edu.in).

This work was supported by the Department of Computer Science at RV College of Engineering.

ABSTRACT Deploying real-time object detection on resource-constrained edge hardware such as the Raspberry Pi presents a fundamental challenge: sustained inference at a fixed configuration inevitably drives the processor into thermal throttling, causing frame rates to collapse and detections to be missed at safety-critical moments. Static pipelines offer no mechanism to reconcile the gap between peak computational demand and the hardware's available thermal headroom. This paper presents a Learned Adaptive Inference System that treats edge deployment as a continuous hardware-aware control problem. The system monitors five live telemetry signals—CPU utilization, RAM consumption, core temperature, instantaneous frame rate, and per-frame latency—and uses these to drive a lightweight neural controller that continuously outputs the optimal input resolution and frame-skip interval for a YOLOv8n pothole detection pipeline.

The system is evaluated on a real-world pothole detection task using a YOLOv8n model deployed on a Raspberry Pi 5, where the thermal constraints of the hardware make adaptive inference particularly consequential.

The learned controller reduces average input resolution from 640 px to 400 px under load, yielding a 61.0% reduction in pixel-level compute with only a 1.8% drop in F1-score. Counter-intuitively, operating at reduced resolution increases detection precision by 1.2% by suppressing false positives caused by road texture and shadow artifacts. These results demonstrate that offline reinforcement learning provides a principled, low-overhead path to thermally stable, continuous pothole detection on edge devices without any hardware modification or model retraining.

INDEX TERMS Adaptive Inference, YOLOv8, Edge Computing, Offline Reinforcement Learning, Thermal Management, Pothole Detection, Raspberry Pi, Dynamic Resolution Scaling, Frame Skipping, Telemetry-Driven Control, Resource-Constrained Inference, IoT Edge Deployment.

I. INTRODUCTION

THE deployment of Internet of Things (IoT) devices and autonomous systems has grown significantly, creating a critical need for performing complex computer vision tasks directly on resource-constrained edge devices. Among these tasks, real-time object detection plays an important role in applications requiring timely environmental perception and

decision-making. The YOLO family of architectures, particularly YOLOv8, has gained significant attention for its favorable balance between detection accuracy and inference speed, making it well suited for real-time object detection on edge platforms. However, deploying deep learning-based object detection models on edge devices presents significant challenges due to inherent hardware limitations, includ-

ing constrained computational resources, memory capacity, power availability, and thermal constraints. These challenges become particularly relevant in continuous real-time applications such as pothole detection, where the detection model must process a persistent stream of visual data while operating within the limited resources of the edge device.

Edge devices such as the Raspberry Pi 5 operate under substantially tighter computational, memory, power, and thermal constraints than conventional desktop or server-class systems. These limitations become particularly significant when deep learning-based object detection is performed continuously on live video streams. A YOLOv8 model executed at a fixed input resolution and frame-processing rate imposes a relatively constant computational workload, which can lead to sustained processor utilization and increasing thermal load during prolonged operation. As the device approaches its thermal limits, frequency throttling can reduce available computational performance, resulting in increased per-frame latency, reduced frame rates, and missed detections. Consequently, a static inference configuration cannot reliably maintain real-time performance under varying hardware conditions, motivating the need for an inference mechanism that dynamically adapts its computational workload to the available resources and thermal headroom.

In practical road-monitoring applications, pothole detection systems may need to operate continuously on edge devices deployed on vehicles for extended periods and under varying environmental and operating conditions. A YOLO-based detection model running continuously on such hardware cannot assume that the available computational and thermal resources will remain at their maximum capacity throughout the entire deployment. Sustained inference at a fixed configuration can increase processor utilization and thermal load, potentially leading to performance degradation through increased latency, reduced frame rates, and thermal throttling. Consequently, maintaining reliable real-time pothole detection requires the inference workload to adapt to the current state of the edge device rather than relying on a fixed configuration. Dynamically adjusting the computational workload according to live hardware and inference conditions can therefore help sustain real-time operation and extend the effective operating period of the detection system under resource-constrained conditions.

A. THE LIMITATION OF STATIC PIPELINES

Current deployment strategies generally treat inference pipelines as static configurations. An engineer selects a fixed input resolution, such as 640×640 , and frames from the camera sensor are then processed sequentially. While this approach theoretically yields consistent inference behavior under ideal conditions, it fails to account for the dynamic physical state of the hardware.

The static paradigm assumes that the edge device can consistently dissipate heat and sustain peak processing speeds. However, when thermal throttling occurs, the software continues demanding a fixed computational workload while

the hardware's available computational capacity diminishes. This mismatch can result in reduced frame rates, increased latency, missed detections, and, in the worst case, system failure. A robust edge AI system must monitor its operational state and dynamically adjust its computational demands to preserve overall stability and responsiveness.

B. PROPOSED SOLUTION: LEARNED ADAPTIVE INFERENCE

To address these challenges, a Learned Adaptive Inference System is introduced. This system transforms the standard YOLOv8 inference pipeline into an adaptive process that continuously responds to the live hardware state. The pipeline exposes two tunable control parameters at runtime: the input resolution $imgsz \in [256, 640]$ (constrained to multiples of 32) and the frame-skip probability $p_{skip} \in [0, 1]$, which determines the proportion of incoming frames that bypass the forward pass entirely. Together, these parameters form a continuous action space that allows fine-grained, real-time trade-offs between detection fidelity and computational demand.

A dedicated monitoring module samples five hardware telemetry signals asynchronously at 2 Hz—CPU utilization, RAM utilization, core temperature, smoothed inference frame rate, and per-frame latency—maintaining a thread-safe state vector s_t accessible to the inference loop in $\mathcal{O}(1)$ time. A lightweight neural Controller reads s_t at each decision step and emits a continuous action pair $(imgsz_t, p_{skip,t})$, dynamically balancing detection quality against the available thermal and compute headroom of the edge device.

The resource management problem is formulated as a Markov Decision Process (MDP), enabling the system to be developed and evaluated through a rigorous offline reinforcement learning workflow:

- 1) **Baseline Profiling:** Characterizing the thermal failure modes of a fixed 640×640 static pipeline under sustained load, establishing the performance upper bound and the conditions under which degradation occurs.
- 2) **Continuous Action Space Exploration:** An exploration controller randomly samples continuous $(imgsz, p_{skip})$ pairs across the full parameter range, executing inference and logging state-action-reward-next-state transition tuples to build the offline RL dataset.
- 3) **Heuristic Control (Baseline):** A rule-based controller that selects from three discrete configurations— $M_0 = (640, 0)$, $M_1 = (480, 1)$, $M_2 = (320, 2)$ —using CPU load, RAM utilization, and temperature thresholds. This serves as a strong interpretable baseline against which the learned policy is evaluated.
- 4) **Offline RL via Filtered Behavioral Cloning:** The collected transitions are filtered to retain the top 30% by reward. A Continuous Actor network is then trained offline via supervised regression on these high-quality transitions, learning to map s_t directly to continuous $(imgsz, p_{skip})$ actions. This offline reinforcement

learning approach produces a policy that has internalized the hardware's thermal dynamics without requiring any online interaction during training.

C. STRUCTURE OF THE PAPER

The remainder of this paper is organized as follows: Section II reviews existing research on edge object detection, thermal management, and dynamic inference. Section III describes the proposed system architecture, including hardware monitoring and inference execution layers. The step-by-step methodology, progressing from baseline profiling to reinforcement learning, is detailed in Section IV. Experimental results are presented in Section V, followed by a discussion of the findings and limitations in Section VI. Finally, Section VII concludes the paper and suggests directions for future research.

II. RELATED WORK

The fields of edge computing, deep learning, and adaptive resource management have evolved rapidly, with recent research focusing on improving detection accuracy while reducing computational, energy, and thermal costs. The present work builds upon these research directions while addressing the specific challenge of maintaining stable real-time object detection performance under prolonged thermal stress on resource-constrained edge devices.

A. POTHOLE DETECTION ON EDGE DEVICES

Deep learning-based object detection has become an effective approach for automated road-damage and pothole detection. Recent datasets and studies have established the use of deep learning for road-surface assessment, while improved YOLO-based architectures have demonstrated promising accuracy for pothole detection in diverse road environments [1], [2], [5], [6]. In particular, recent YOLOv8-based approaches have investigated improved feature extraction, segmentation, and detection performance for complex road conditions [3], [11], [12]. Lightweight road-distress detection models have also been explored for deployment on mobile and resource-constrained platforms [6].

The suitability of these models for edge deployment has been further demonstrated through real-time object detection studies on embedded hardware [13], [14]. However, most existing approaches primarily optimize the model architecture or inference implementation for computational efficiency. Such static optimizations do not directly address the performance degradation that can occur when an edge device performs continuous inference and accumulates heat over extended periods. The proposed system instead introduces a runtime pipeline-level adaptation mechanism that dynamically adjusts input resolution and processing frequency according to the device's thermal state, allowing the same YOLOv8n model to operate under changing thermal conditions without retraining or replacing the model.

B. DYNAMIC NEURAL NETWORKS AND ADAPTIVE INFERENCE

Dynamic Neural Networks provide a prominent approach for reducing inference computation by adapting the amount of processing performed for each input. BranchyNet introduced early-exit branches that allow samples with sufficiently confident predictions to terminate inference before reaching the full network depth, thereby reducing computational cost [23]. Similarly, dynamic channel pruning selectively suppresses computational channels to reduce the workload associated with neural network inference [21].

More recent work has extended adaptive inference to object detection. AdaDet introduces an early-exit mechanism specifically for object detection and uses detection uncertainty to determine whether an input can terminate computation early [17]. Other approaches have investigated adaptive inference directly on edge devices, dynamically adjusting inference behavior according to available computational resources [18]. Energy-aware approaches have further combined neural-network optimization with hardware-level frequency scaling to reduce energy consumption while satisfying latency constraints [8].

Although these techniques can provide substantial computational savings, many require architectural modifications, additional exit branches, specialized policies, or retraining. Their computational savings may also depend on the complexity of individual inputs. In contrast, the proposed approach operates at the inference-pipeline level: input resolution and frame-processing frequency are dynamically adjusted while retaining a single, unmodified YOLOv8n model. This allows the adaptation mechanism to be applied without retraining and provides direct control over the workload imposed on the edge device.

C. THERMAL MANAGEMENT IN EDGE COMPUTING

Thermal management is an important consideration for embedded platforms executing computationally intensive workloads. Conventional operating-system-level thermal management can rely on frequency scaling when device temperatures approach critical limits, which can lead to reduced computational performance and violations of real-time requirements. Maity et al. investigated thermal management for heterogeneous CPU-GPU embedded platforms and demonstrated a predictive control strategy that considers workload scheduling and frequency tuning to reduce peak temperature while maintaining real-time requirements [19].

Recent energy-efficient inference research has also explored the interaction between neural-network computation and hardware power management. Dai et al. proposed a software-hardware co-design approach that combines neural-network optimization, early exits, and frequency scaling to reduce energy consumption while satisfying latency constraints [8]. These studies demonstrate the importance of considering both computational workload

and hardware operating conditions when deploying deep-learning workloads on edge devices.

However, the aforementioned approaches primarily focus on hardware scheduling, frequency management, or energy optimization. The proposed system instead establishes a direct feedback loop between hardware temperature telemetry and the computer-vision inference pipeline. When the measured temperature increases, the controller reduces the computational workload by modifying inference resolution and frame-processing frequency before severe thermal throttling occurs. This application-level strategy is designed specifically for continuous video inference and aims to maintain stable detection performance while keeping the device within a safe thermal operating range.

D. REINFORCEMENT LEARNING AND EDGE-CLOUD RESOURCE OPTIMIZATION

Reinforcement Learning (RL) has increasingly been investigated for resource management in edge-computing environments. Recent surveys have examined RL-based approaches for mobile edge computing, including resource allocation, task scheduling, and computation offloading [7]. For example, dependency-aware reinforcement learning has been used to formulate dynamic task-offloading decisions as an optimization problem involving changing computational and communication resources [9].

Deterministic task-offloading approaches have also been investigated for the IoT-edge-cloud continuum. Aghababaiyan et al. proposed a deterministic task-offloading and resource-allocation scheme that jointly considers communication and computing resources across IoT devices, edge servers, and cloud infrastructure [25]. Their approach prioritizes completing tasks within their deadlines rather than minimizing the execution latency of each individual task, thereby providing deterministic service levels under resource-constrained conditions. Such approaches demonstrate that computation can be dynamically distributed across the IoT-edge-cloud continuum when communication and remote computational resources are available.

Adaptive inference has also been explored as a means of dynamically modifying the computational resources allocated to deep neural network workloads. Edge-device adaptive inference approaches demonstrate that inference behavior can be modified according to device conditions and workload requirements [18]. These studies motivate the use of feedback-driven decision-making for managing computational workloads on resource-constrained edge devices.

The present work differs from task-offloading approaches in its control objective and deployment architecture. Rather than distributing inference tasks across remote computing resources or selecting among multiple neural network architectures, the proposed system adapts the execution parameters of a single YOLOv8n detector locally. Specifically, the learned controller adjusts the input resolution and frame-skipping probability according to real-time hardware and

inference telemetry. This provides workload adaptation without requiring network connectivity or transferring inference workloads to external computing infrastructure. The heuristic controller is retained as an interpretable baseline, while the final learned controller provides continuous adaptation of the inference parameters.

III. SYSTEM ARCHITECTURE

The system architecture is designed to decouple hardware monitoring, the decision-making controller, and the computationally intensive inference execution. This separation ensures that the overhead of adaptivity does not impede the detection pipeline itself.

A. HARDWARE ABSTRACTION AND TELEMETRY LAYER

At the foundation of the system lies a Hardware Abstraction Layer (HAL) that continuously monitors the operational state of the target edge device. The HAL runs in a dedicated background thread, allowing telemetry to be collected without blocking the real-time inference loop. In this work, the proposed system is implemented and evaluated on a Raspberry Pi 5 as the target resource-constrained edge platform.

The monitored hardware metrics comprise CPU utilization (U_{cpu}), RAM utilization (U_{ram}), and core temperature (T_{core}). These are combined with two inference-derived metrics: the smoothed frame rate (FPS_{avg}) and per-frame latency (L_t). Together, these five signals form the complete state vector s_t supplied to the adaptive controller:

$$s_t = [U_{cpu}, U_{ram}, T_{core}, FPS_{avg}, L_t]. \quad (1)$$

The HAL samples these signals asynchronously at a fixed interval of 500 ms (2 Hz) and stores the latest telemetry snapshot in a thread-safe in-memory structure. The inference pipeline can therefore access the current state without waiting for a telemetry update, introducing negligible monitoring overhead to the detection process.

B. THE ADAPTIVE CONTROLLER

The adaptive controller serves as the decision-making component of the system, receiving the latest telemetry state from the Hardware Abstraction Layer. At each control step, the controller processes the five-dimensional state vector $s_t = [U_{cpu}, U_{ram}, T_{core}, FPS_{avg}, L_t]$ and produces a continuous action consisting of the raw image-resolution and frame-skipping parameters.

The learned controller is implemented as a lightweight Continuous Actor network with a $5 \rightarrow 128 \rightarrow 128 \rightarrow 2$ multilayer perceptron (MLP). ReLU activations are used in the hidden layers, while a Tanh activation at the output constrains the two raw action values to the range $[-1, 1]$. These outputs are subsequently mapped to the operational inference parameters: the image resolution $imgsz_t$ is mapped to the range $[256, 640]$ and constrained to multiples of 32, while the frame-skipping probability $p_{skip,t}$ is mapped to $[0, 1]$.

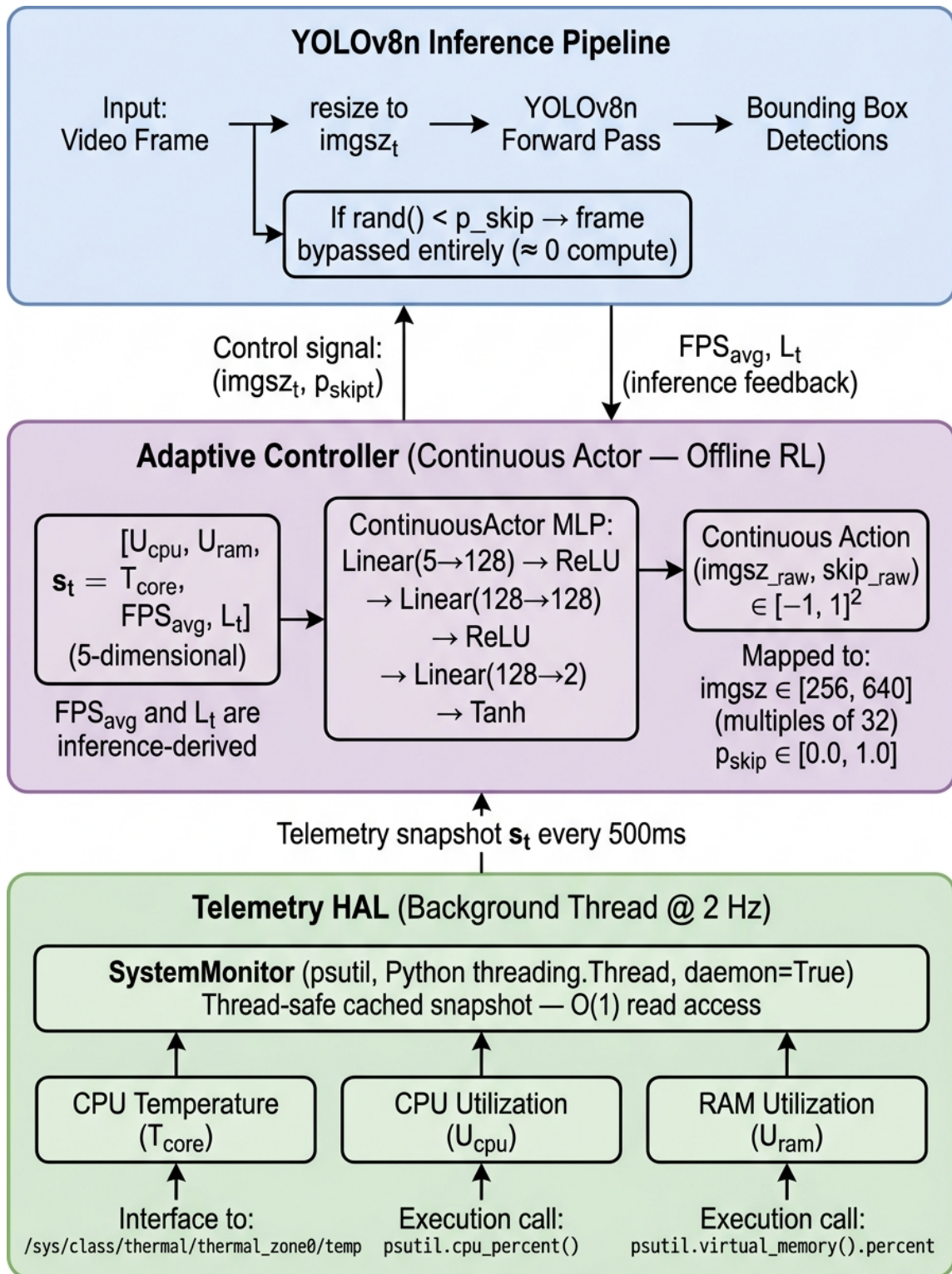


FIGURE 1. Comprehensive System Architecture showing the asynchronous interaction between the Hardware Monitoring Thread, the State Space Aggregator, the Adaptive Controller (Heuristic/Learned), and the YOLOv8 Inference Pipeline.

The controller is trained offline using reward-filtered behavioral cloning. During the data-collection phase, continuous actions are explored on the target edge platform and the resulting state-action-reward transitions are recorded. The

highest-reward transitions are then used as demonstrations for training the Continuous Actor to learn a direct mapping from the observed hardware and inference state to an appropriate inference configuration. Consequently, no online learn-

ing or parameter updates are performed during deployment.

At runtime, the resulting action pair $(imgsz_t, p_{skip,t})$ is passed to the inference pipeline. This allows the computational workload to be continuously adjusted according to the current operating state of the edge device rather than relying on a fixed inference configuration.

C. THE YOLOV8N INFERENCE PIPELINE

The visual processing stage is implemented using the Ultralytics YOLOv8n detector. The inference pipeline receives video frames together with the current control parameters $(imgsz_t, p_{skip,t})$ generated by the adaptive controller. For frames selected for processing, the input image is resized to the controller-selected resolution $imgsz_t$ and passed through the YOLOv8n forward pass to produce object detections. The resulting predictions are subsequently post-processed to obtain the final bounding-box detections.

The frame-skipping mechanism is probabilistic, with the controller output $p_{skip,t}$ defining the probability that an incoming frame is bypassed. Thus,

$$P(\text{skip}_t) = p_{skip,t}, \quad P(\text{process}_t) = 1 - p_{skip,t}. \quad (2)$$

Frames selected for skipping bypass the inference pipeline, avoiding the computational cost of preprocessing, forward inference, and post-processing. Frames selected for processing are resized according to the current $imgsz_t$ configuration and passed to the YOLOv8n detector.

The inference pipeline also provides feedback to the adaptive controller in the form of the smoothed frame rate FPS_{avg} and per-frame latency L_t . These inference-derived measurements are incorporated into the telemetry state used at the subsequent control step. The resulting closed-loop architecture allows the computational workload of the YOLOv8n pipeline to be continuously adjusted according to the current hardware and inference conditions rather than relying on a fixed inference configuration.

IV. METHODOLOGY

The adaptive system was developed through a rigorous, multi-phase sequence. This systematic progression ensures that the final learned policy is evaluated against well-understood baselines rather than arbitrary benchmarks.

A. PHASE 0: BASELINE STATIC INFERENCE

The objective of Phase 0 is to quantify the performance limitations of a fully static inference pipeline on the target edge hardware, providing an empirical baseline that motivates the need for adaptive control.

In this phase, the system operates as a standard static pipeline. YOLOv8n is configured with a fixed input resolution of 640×640 pixels and FP32 precision, with no frame skipping. Every incoming frame is processed unconditionally, resulting in a consistently high computational workload.

During execution, the system records the timestamp, frame rate, per-frame latency, CPU utilization, RAM utilization, core temperature, number of detections, average detection

confidence, and cumulative frame-drop rate for each processed frame.

On the Raspberry Pi 5 target platform, the static pipeline demonstrates limited real-time performance. Across the evaluation run, the achieved frame rate ranges from 1.8 to 4.01 FPS, while per-frame latency ranges from approximately 245 to 665 ms. The cumulative frame-drop rate remains high, ranging from 85.73% to 96.88% and settling at approximately 87.04% toward the end of the run. Core temperature increases from approximately 50°C to a maximum of 60.6°C , while RAM utilization remains relatively stable around 30%. The observed temperature range provides no evidence of thermal throttling during this particular run, indicating that the primary limitation of the static configuration is the computational throughput required for full-resolution YOLOv8n inference on the Raspberry Pi 5.

The resulting behavior is illustrated in Fig. 2. The figure shows the low sustained frame rate and high inference latency, together with substantial frame loss throughout the run. These results demonstrate that a fixed 640×640 inference configuration is unsuitable for sustained real-time operation under the tested edge-device conditions. Reducing the computational workload through lower input resolution, frame skipping, or a combination of both therefore provides a basis for the adaptive inference strategy developed in the subsequent phases.

B. PHASE 1: ACTION SPACE DEFINITION AND CONTROLLED ADAPTATION

Phase 0 establishes that a static 640×640 pipeline imposes a substantial computational workload on the Raspberry Pi 5. Phase 1 therefore defines the controllable parameters of the inference pipeline and experimentally characterizes their effect on computational load and inference performance.

The action space is based on two independent workload-control mechanisms: input resolution and inference frequency. The input resolution $imgsz$ determines the spatial dimensions of the tensor presented to YOLOv8n, while the frame-skipping parameter controls how frequently incoming frames are processed.

Rationale for Resolution Scaling

The computational cost of convolutional processing depends strongly on the spatial dimensions of the input tensor. Reducing the input resolution therefore provides a direct mechanism for reducing the computational workload without modifying the YOLOv8n model architecture or its learned parameters. In particular, reducing the input from 640×640 to 320×320 reduces the input spatial area by 75%, providing a substantial reduction in the amount of spatial computation performed by the network.

Rationale for Frame Skipping

Resolution scaling reduces the computational cost of each processed frame, whereas frame skipping reduces the number

Run Overview: logs_rpi5.csv

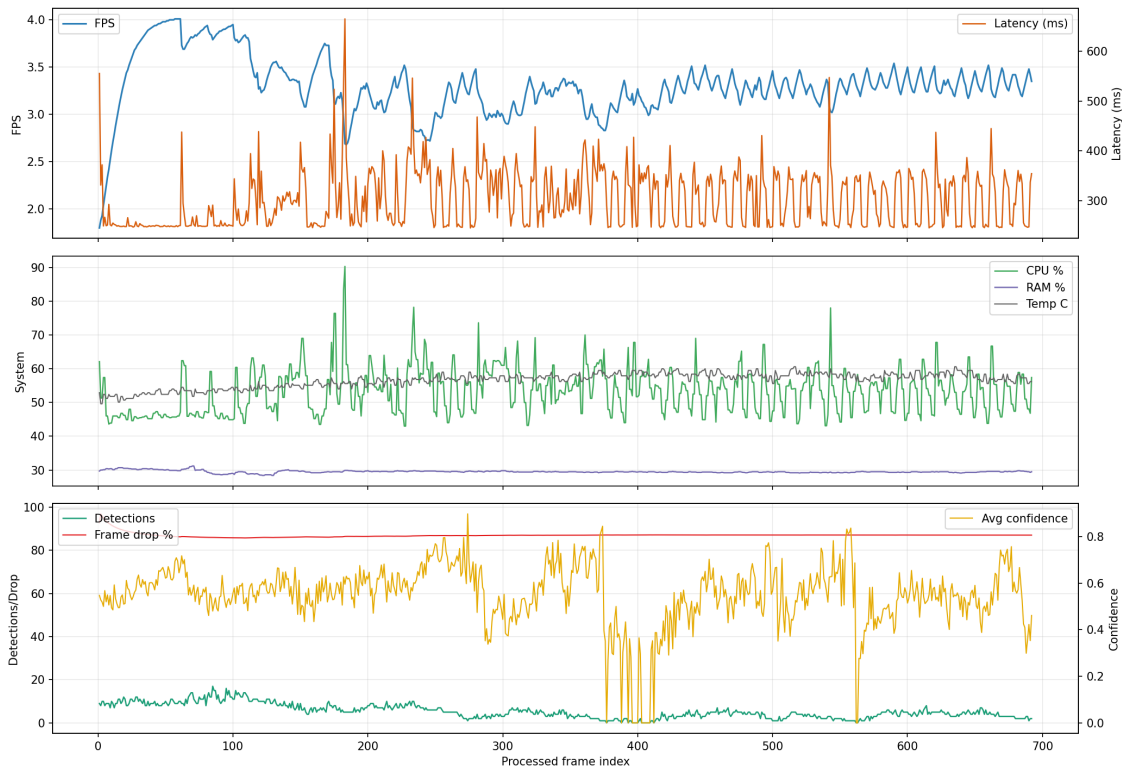


FIGURE 2. Phase 0 static baseline on the Raspberry Pi 5 using 640×640 input resolution, FP32 precision, and no frame skipping. *Top:* FPS and per-frame latency over the evaluation run. *Middle:* CPU utilization, RAM utilization, and core temperature. *Bottom:* detection count, average detection confidence, and cumulative frame-drop rate. The static configuration exhibits low sustained frame rates, substantial inference latency, and a high frame-drop rate, demonstrating the limitations of fixed full-resolution inference under the resource constraints of the target edge platform.

of frames on which inference is performed. These mechanisms therefore provide complementary forms of workload control. For the heuristic baseline, three discrete configurations are defined:

- $M_0 = (640, 0)$: full-resolution inference with no frame skipping.
- $M_1 = (480, 1)$: reduced-resolution inference with every alternate frame bypassed.
- $M_2 = (320, 2)$: reduced-resolution inference with two out of every three frames bypassed.

These configurations span progressively lower computational workloads and are used to characterize the resolution-throughput trade-off on the target hardware.

During Phase 1, the feedback controller is disabled and each configuration is applied for a fixed duration. The resulting frame rate, latency, CPU utilization, and temperature are recorded to quantify the computational and thermal characteristics of each operating point. Detection count and average detection confidence are also recorded to assess the effect of workload reduction on detection behavior. These measurements provide the empirical basis for the heuristic thresholds introduced in the subsequent phase.

Phase 1 Observations and Trade-off Analysis

The experimental results demonstrate a clear trade-off between computational workload and detection performance, as shown in Fig. 3. As the operating configuration moves from M_0 toward the lower-workload configurations, the average inference throughput increases and the tail latency decreases. The measured average FPS increases from approximately 30.3 FPS for M_0 to 50.2 FPS for M_1 and 51.5 FPS for M_2 . Similarly, the P95 latency decreases from approximately 367 ms for M_0 to 202 ms for M_1 and 147 ms for M_2 .

However, the improvement in computational performance is accompanied by a reduction in the number of detections generated during the observation period. The average number of detections decreases from approximately 8.6 for M_0 to 3.7 for M_1 and 2.5 for M_2 . The average detection confidence also decreases moderately, from approximately 0.575 for M_0 to 0.508 for M_1 and 0.485 for M_2 . Thus, more aggressive workload reduction improves throughput and latency, but also reduces the temporal frequency of detection and can affect detection confidence.

These observations show that the three operating configurations represent different workload-performance trade-offs rather than uniformly improving operating points. In par-

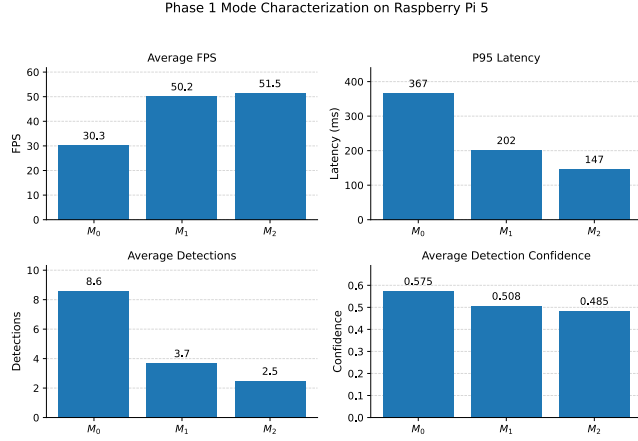


FIGURE 3. Phase 1 mode characterization on the Raspberry Pi 5. The lower-workload configurations increase average FPS and reduce P95 latency, but also reduce the average number of detections and moderately decrease average detection confidence.

ticular, M_1 provides a substantial throughput improvement over M_0 while maintaining a moderate reduction in detection confidence, whereas the transition from M_1 to M_2 provides a further reduction in P95 latency but only a small increase in average FPS and a further reduction in detection frequency. This demonstrates that selecting an operating point involves balancing computational efficiency against detection availability and performance.

The results therefore motivate adaptive workload control rather than the use of a single fixed configuration. Under changing device conditions, a controller can select an appropriate operating point according to the current computational and inference state. The Phase 1 measurements therefore provide the empirical motivation for the adaptive-control stages that follow.

The discrete configurations are therefore used only for empirical characterization and as an interpretable heuristic baseline. The final learned controller instead operates over the continuous action parameters $(imgsz_t, p_{skip,t})$, allowing it to select intermediate operating points rather than being restricted to the three discrete configurations evaluated in Phase 1. The observed workload–performance trade-off provides the motivation for learning such a continuous adaptation policy.

C. PHASE 2 & 3: DETERMINISTIC ADAPTIVE CONTROLLER

Phase 2 introduces closed-loop telemetry by continuously monitoring U_{cpu} , U_{ram} , and T_{core} during inference. Phase 3 connects this telemetry to a deterministic rule-based controller that selects an inference mode at each decision step.

The controller maps the current resource utilization directly to one of the three modes defined in Phase 1. The decision is based on CPU and RAM utilization as follows:

Algorithm 1 Deterministic Adaptive Controller

Require: U_{cpu}, U_{ram}

Ensure: $m_{next} \in \{M_0, M_1, M_2\}$

```

1: if  $U_{cpu} > 80.0$  or  $U_{ram} > 85.0$  then
2:    $m_{next} \leftarrow M_2$            ▷ Severe workload reduction
3: else if  $U_{cpu} > 50.0$  then
4:    $m_{next} \leftarrow M_1$        ▷ Moderate workload reduction
5: else
6:    $m_{next} \leftarrow M_0$        ▷ Full-resolution inference
7: end if

```

The thresholds were selected from the operating ranges observed during Phase 0 and Phase 1. Since sustained inference produced CPU utilization within approximately 45–90%, 50% was used to trigger moderate workload reduction, while 80% was selected for severe load conditions. The 85% RAM threshold provides an additional safeguard against high memory utilization. The controller is stateless, with each decision made independently of the previous mode; consequently, no hysteresis or recovery timer is applied.

This deterministic controller demonstrates that adaptive workload control can be achieved using simple and auditable rules. However, its fixed thresholds restrict the controller to three predefined operating points. The subsequent learned controller therefore replaces these discrete rules with a continuous policy over $(imgsz_t, p_{skip,t})$, enabling adaptation across a broader range of workload conditions.

D. PHASE 4: MDP FORMULATION AND OFFLINE DATA COLLECTION

The adaptive inference problem is formulated as a Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T} \rangle$.

State Space

The state vector captures the five telemetry signals that most directly characterize the current computational and thermal operating condition of the Raspberry Pi 5:

$$s_t = [U_{cpu}, U_{ram}, T_{core}, FPS_{avg}, L_t] \quad (3)$$

U_{cpu} and U_{ram} quantify the current computational load and memory pressure. T_{core} captures the thermal state of the SoC. FPS_{avg} reflects the achieved inference throughput as observed by the pipeline, and L_t captures per-frame inference responsiveness. Together these five signals provide a complete characterization of the hardware operating condition without requiring GPU-specific sensors unavailable on the Raspberry Pi 5.

Continuous Action Space

Rather than restricting the learned controller to the three discrete configurations used in Phase 1 for characterization (M_0, M_1, M_2), Phase 4 defines a continuous action space that allows the policy to select intermediate resolutions and arbitrary skip probabilities. The controller outputs a raw

action $a_t \in [-1, 1]^2$, which is mapped to:

$$\text{imgsz}_t = \text{round}_{32} \left(256 + \frac{a_t^{(0)} + 1}{2} (384) \right),$$

$$\text{imgsz}_t \in \{256, 288, \dots, 640\} \quad (4)$$

$$p_{\text{skip},t} = \frac{a_t^{(1)} + 1}{2}, \quad p_{\text{skip},t} \in [0, 1] \quad (5)$$

The resolution range [256, 640] spans the practical workload range of YOLOv8n on the target hardware; values below 256 were not used as they produced negligible detections. Multiples of 32 are required by the YOLO backbone's stride constraints.

The continuous skip probability $p_{\text{skip},t}$ allows the controller to regulate inference frequency continuously rather than selecting from fixed skip counts. At each step, an incoming frame is bypassed with probability $p_{\text{skip},t}$ and passed to YOLOv8n otherwise.

Decision Interval

The policy issues a new action every 5 processed frames. This interval was selected as a practical engineering compromise: it provides a short but stable observation window for the telemetry to settle, avoids excessive policy updates that would add control overhead to the inference loop, and ensures the controller remains responsive to changing system conditions. No formal optimality claim is made for this value.

Reward Function

The reward function encodes the engineering objectives:

$$R_t = 10\hat{a}_t + 0.5 FPS_t - 0.1 L_t - 50 \cdot \mathbf{1}[T_{\text{core}} > 80] - 10 \cdot \mathbf{1}[U_{\text{cpu}} > 90] \quad (6)$$

where $\hat{a}_t$ is the maximum detection confidence in the current window (proxy for detection quality), and all terms are evaluated at s_{t+1} .

The coefficient priorities reflect the intended engineering hierarchy: detection quality is strongly rewarded ($\times 10$) as the primary objective; throughput is encouraged ($\times 0.5$) as a secondary objective; per-frame latency is penalized ($\times 0.1$) to discourage configurations that are accurate but unresponsive; a large flat penalty (-50) is applied when $T_{\text{core}} > 80^\circ\text{C}$ to strongly discourage thermal overload; and a moderate penalty (-10) is applied when $U_{\text{cpu}} > 90\%$ to discourage CPU saturation. The coefficient values represent the selected engineering priorities.

Offline Data Collection

Data collection uses a uniform random exploration policy: actions are sampled i.i.d. from $\text{Uniform}(-1, 1)^2$, providing broad coverage of the continuous action space without any bias toward previously seen states. The pipeline records (s_t, a_t, R_t, s_{t+1}) transitions for offline training. Collection is

bounded to a practical budget of 60 minutes or 20,000 transitions, whichever is reached first. This budget was selected as a feasible on-device collection window.

E. PHASE 5: OFFLINE POLICY LEARNING

With the continuous action space and offline transition dataset established, Phase 5 focuses on extracting an effective control policy. This is achieved by filtering the collected data for high-performing states and training a lightweight neural network to emulate those actions.

Filtered Behavioral Cloning

Phase 5 trains the learned controller entirely offline using reward-filtered behavioral cloning (FBC) [16]. The offline dataset is first filtered to retain only the top 30% of transitions by reward—those exceeding the 70th-percentile reward threshold. This filtering step biases the training distribution toward actions associated with better observed workload-performance trade-offs, preventing the policy from imitating low-reward or thermally unsafe operating points. The retained transitions constitute the training set $\mathcal{D}_{\text{top30\%}}$. This is an engineering data-selection strategy.

Policy Architecture

The learned control policy is implemented as a compact Multi-Layer Perceptron (MLP) with five inputs (matching the state dimensionality) and two continuous outputs:

$$\pi_\theta : \mathbb{R}^5 \xrightarrow{128\text{-ReLU}} \mathbb{R}^{128} \xrightarrow{128\text{-ReLU}} \mathbb{R}^2 \xrightarrow{\tanh} [-1, 1]^2 \quad (7)$$

The architecture is intentionally lightweight: a low-dimensional state-to-action mapping (5 inputs $\rightarrow$ 2 outputs) does not require deep or wide networks. Two hidden layers of 128 units provide sufficient nonlinear capacity for this low-dimensional regression task while maintaining a small parameter count suitable for Raspberry Pi 5 deployment. The Tanh output layer naturally constrains the raw action to $[-1, 1]^2$, directly matching the action space convention used during data collection (Sec. IV-D).

Input Normalization

All five state features are z-score normalized before being passed to the network:

$$\tilde{s}_t = (s_t - \mu_{\text{train}}) / \sigma_{\text{train}} \quad (8)$$

where μ_{train} and σ_{train} are the per-feature mean and standard deviation computed from $\mathcal{D}_{\text{top30\%}}$. Normalization places features with heterogeneous numerical scales (U_{cpu} : 0–100%, L_t : 0–300 ms, T_{core} : 30–80 $^\circ\text{C}$, etc.) on a comparable numerical range, preventing large-magnitude features from dominating gradient updates. The normalization statistics are saved alongside the policy weights and applied identically at deployment time.

Training Objective and Hyperparameters

The policy is trained to reproduce the continuous actions of the retained high-reward transitions via mean-squared error regression:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}_{top30\%}} [\|\pi_{\theta}(\tilde{s}) - a\|^2] \quad (9)$$

Mean Squared Error (MSE) is appropriate here because the goal is direct regression from normalized state to a continuous raw action in $[-1, 1]^2$. Training runs for 150 epochs using the Adam optimizer (learning rate 10^{-3} , batch size 64). These hyperparameters were selected as standard settings for this scale of supervised regression.

F. PHASE 6: DEPLOYMENT AND INFERENCE

The trained network is exported as a TorchScript module, enabling portable, dependency-free loading on the Raspberry Pi 5 without requiring a full PyTorch training environment. At deployment, a single forward pass of the $5 \rightarrow 128 \rightarrow 128 \rightarrow 2$ network completes in under 1 ms on the Raspberry Pi 5 CPU, imposing negligible overhead on the inference loop.

The deployed policy is fully frozen: no gradient updates, no online exploration, and no parameter modifications occur during deployment. The system performs only deterministic forward inference using the pre-trained actor weights. This ensures that the deployed controller behavior is consistent, auditable, and free of any online learning risk.

V. EXPERIMENTAL RESULTS

The experimental evaluation focuses on the runtime and perception-level effects of adaptive inference relative to a fixed 640×640 baseline. The evaluation considers two complementary aspects: (i) runtime behavior, including throughput, latency, CPU utilization, RAM utilization, and the selected inference resolution, and (ii) detection fidelity on a labeled image set. The reported values below are taken directly from the generated evaluation artifacts.

A. EXPERIMENTAL SETUP

The runtime evaluation compares a fixed-resolution baseline with the learned adaptive inference configuration on the same target evaluation sequence. The baseline uses a fixed 640×640 input resolution, whereas the adaptive configuration dynamically selects the inference resolution and frame-processing behavior according to the learned policy.

For the accuracy evaluation, the two configurations are compared on 51 labeled test images using an Intersection-over-Union (IoU) threshold of 0.5. The baseline uses a fixed 640×640 input resolution, while the adaptive configuration uses a variable input resolution with an average selected image size of approximately 400 pixels, as reported in the evaluation artifact. Precision, recall, F1-score, and the reported mAP@0.5 metric are used to characterize the perception-level trade-off.

The runtime evaluation records inference FPS, per-frame latency, CPU utilization, RAM utilization, and the image

size selected by the adaptive controller. These measurements characterize local inference behavior and do not constitute an end-to-end latency measurement involving network transmission or remote computation.

B. RUNTIME PERFORMANCE OF THE LEARNED ADAPTIVE CONTROLLER

The mean runtime measurements are summarized in Fig. 4. The fixed baseline achieves a mean throughput of 6.3 FPS, whereas the learned adaptive configuration achieves 10.4 FPS, corresponding to an increase of 4.1 FPS. The mean inference latency decreases from 169.6 ms for the baseline to 100.4 ms for the adaptive configuration, a reduction of 69.2 ms.

The CPU and RAM measurements show a different behavior. Mean CPU utilization changes only from 99.9% for the baseline to 99.8% for the adaptive configuration, while mean RAM utilization decreases from 75.1% to 74.2%. Thus, the observed runtime improvement cannot be attributed to a substantial reduction in CPU occupancy. Instead, the results indicate that reducing the computational workload per processed frame through adaptive inference parameters allows the system to complete inference more quickly despite remaining computationally saturated.

The temporal behavior of the two configurations is shown in Fig. 5. The adaptive configuration maintains a higher FPS trajectory over most of the recorded video. The baseline FPS generally remains below the adaptive trajectory, while the adaptive configuration operates predominantly in the approximately 9–12 FPS range after the initial transient. The corresponding latency trace is also generally lower for the adaptive configuration.

The latency traces also exhibit transient spikes in both configurations. Consequently, the results demonstrate an improvement in measured local inference performance but do not establish deterministic latency or elimination of temporal variability. In particular, the reported FPS and latency values describe local inference execution and do not include network transmission, communication queueing, or remote computation delays.

C. DETECTION ACCURACY AND FIDELITY TRADE-OFF

The perception-level comparison is presented in Fig. 6. On the 51-image evaluation set, the baseline obtains an mAP@0.5 value of 69.6%, compared with 67.6% for the adaptive configuration. The corresponding change is a 2.0 percentage point decrease.

Precision increases from 61.2% for the baseline to 62.3% for the adaptive configuration, corresponding to a 1.2 percentage point increase. In contrast, recall decreases from 70.3% to 64.9%, a reduction of 5.4 percentage points. The resulting F1-score decreases from 65.4% to 63.6%, corresponding to a 1.8 percentage point reduction.

These results demonstrate that the runtime improvement is accompanied by a measurable perception-level trade-off. The

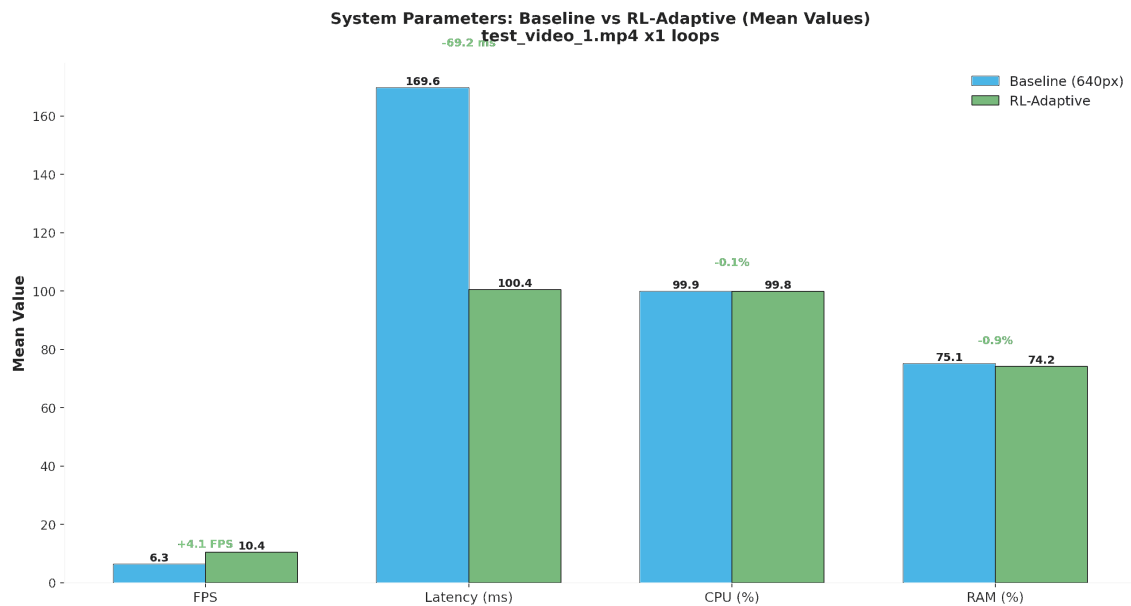


FIGURE 4. Mean runtime parameters for the fixed 640-pixel baseline and the learned adaptive configuration on the target evaluation sequence. The adaptive configuration increases mean throughput from 6.3 to 10.4 FPS and reduces mean latency from 169.6 to 100.4 ms.

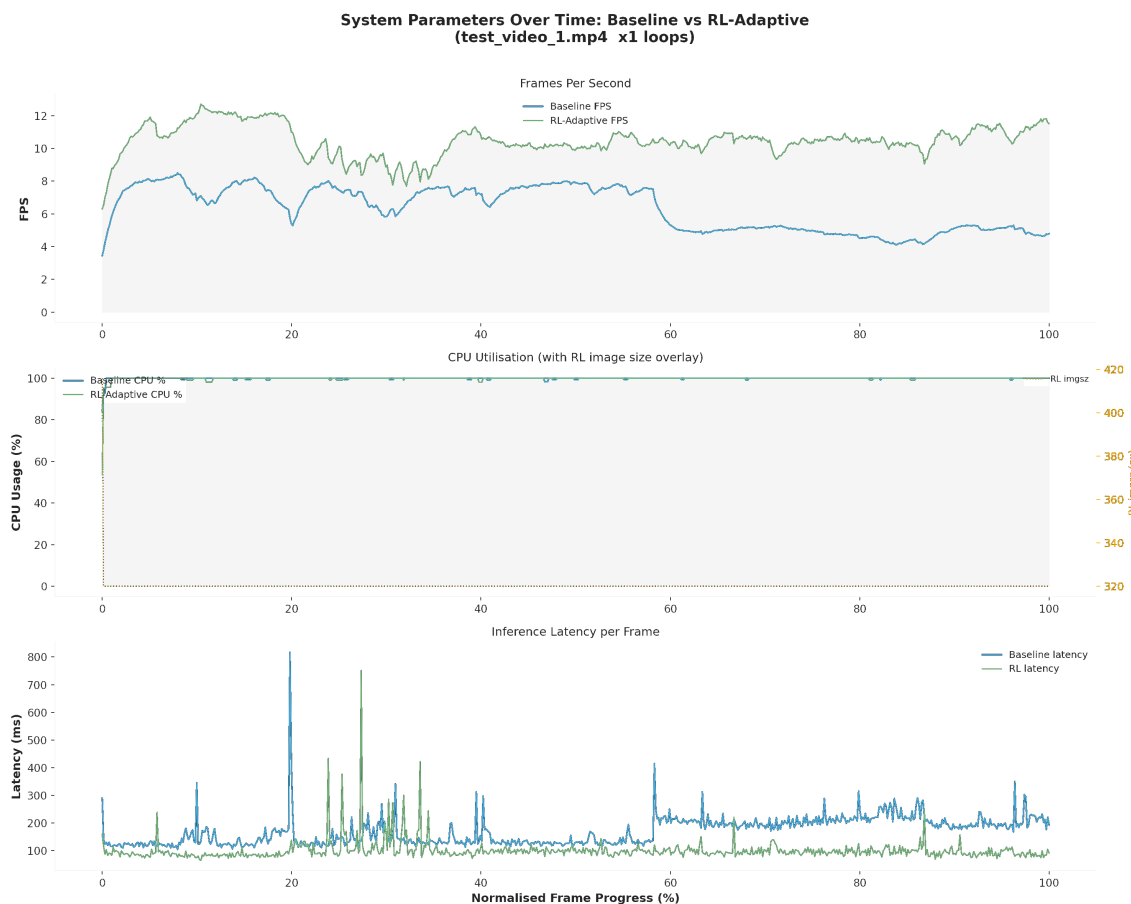


FIGURE 5. Runtime behavior over normalized video progress for the fixed baseline and learned adaptive configurations. The adaptive configuration maintains higher FPS and generally lower per-frame latency. Both configurations remain highly CPU-utilized, while the selected adaptive image size is reduced for most of the run. Occasional latency spikes remain visible, indicating that the measured inference latency should not be interpreted as deterministic end-to-end latency.

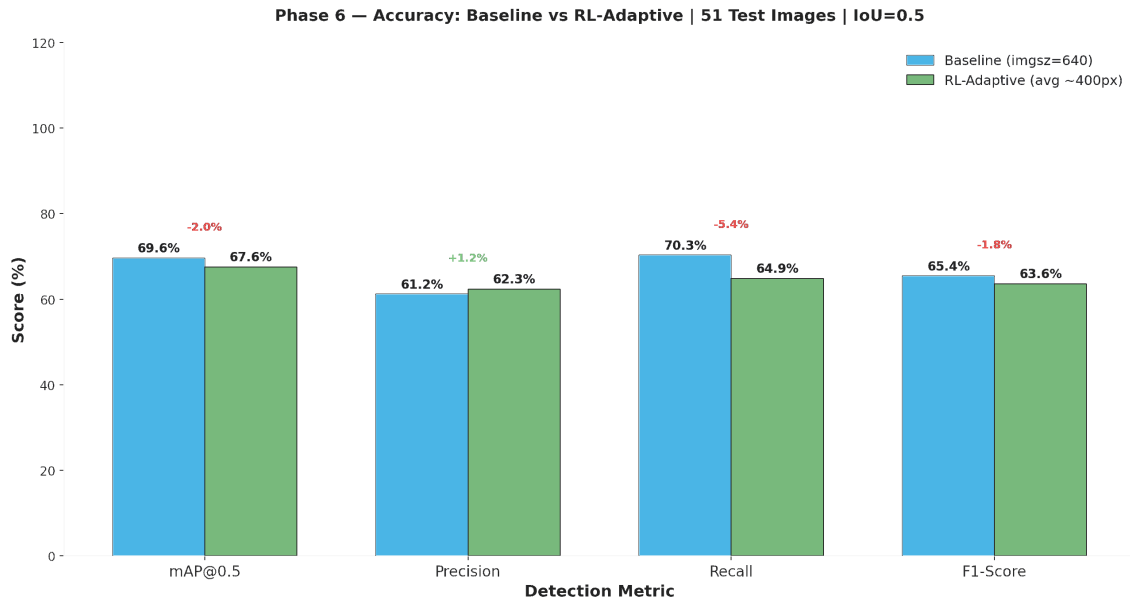


FIGURE 6. Detection accuracy comparison on 51 labeled test images at IoU= 0.5. The baseline uses 640×640 input resolution, whereas the adaptive configuration uses an average selected image size of approximately 400 pixels.

adaptive configuration does not preserve the aggregate detection metrics of the fixed 640-pixel baseline: recall, F1-score, and the reported mAP@0.5 metric are lower, while precision is slightly higher. Therefore, the adaptive controller should be interpreted as a workload–accuracy trade-off mechanism rather than as a method that simultaneously improves all detection metrics.

The accuracy artifact reports an average adaptive image size of approximately 400 pixels. This reduction in spatial resolution is consistent with the observed decrease in recall and F1-score, although the available aggregate results do not establish the effect specifically on small potholes.

D. PER-IMAGE DETECTION BEHAVIOR

Aggregate accuracy metrics can conceal substantial variation between individual images. The per-image comparison in Fig. 7 plots the F1-score obtained by the baseline against the F1-score obtained by the adaptive configuration for individual test images. The diagonal line represents equal performance.

The scatter distribution shows heterogeneous behavior rather than a uniform change in detection quality. Some images lie above or close to the equal-performance diagonal, while others exhibit a lower adaptive F1-score. This indicates that the effect of workload adaptation depends on the visual content of the individual frame.

The color encoding further shows that the adaptive controller can select different input resolutions for different samples. Consequently, the aggregate F1 reduction should not be interpreted as a uniform degradation across all images. Instead, it reflects the combined effect of adaptive resolution selection and the visual characteristics of the evaluated samples.

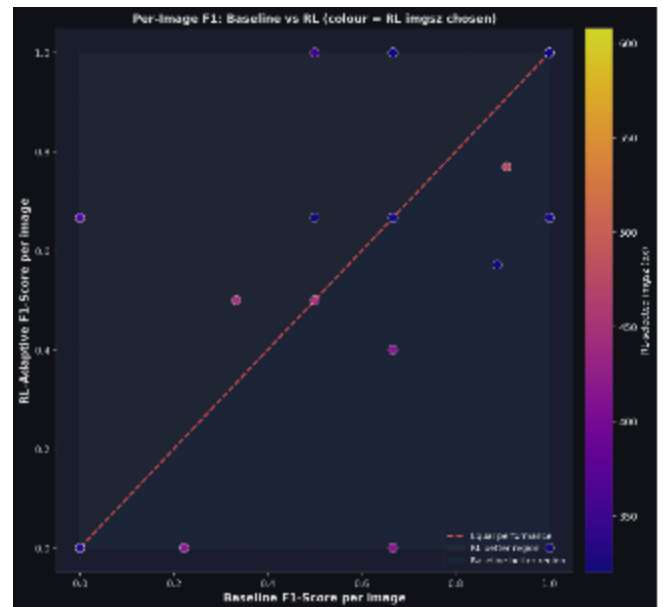


FIGURE 7. Per-image F1-score comparison between the fixed 640-pixel baseline and the learned adaptive configuration. Points above the diagonal indicate higher adaptive F1-score, whereas points below the diagonal indicate lower adaptive F1-score. Point color represents the image resolution selected by the adaptive configuration.

E. RUNTIME-ACCURACY TRADE-OFF

Taken together, Figs. 4–7 show the central trade-off introduced by the learned adaptive inference mechanism. The runtime evaluation shows an increase in mean throughput from 6.3 to 10.4 FPS and a reduction in mean latency from 169.6 to 100.4 ms. Meanwhile, the accuracy evaluation shows decreases of 2.0 percentage points in the reported

mAP@0.5 metric, 5.4 percentage points in recall, and 1.8 percentage points in F1-score, together with a 1.2 percentage point increase in precision.

The runtime measurements therefore indicate that adaptive workload reduction can substantially improve local inference throughput and latency, while the accuracy measurements demonstrate the associated perception trade-off. The results do not support a claim that adaptive inference is universally superior to fixed-resolution inference; rather, they demonstrate that computational workload can be reduced at the cost of measurable changes in detection fidelity.

The CPU measurements provide an additional observation: both configurations operate at approximately 100% mean CPU utilization. Thus, the measured performance improvement is primarily associated with changing the amount of computation performed per inference and the number of frames processed, rather than with maintaining a lower overall CPU utilization.

F. SCOPE OF THE RUNTIME MEASUREMENTS

The reported FPS and latency measurements are local inference measurements. They characterize execution of the vision pipeline on the evaluated edge system and do not include network transmission or edge/cloud queueing effects. The temporal plots also demonstrate that individual latency spikes remain present even when the adaptive configuration achieves a lower mean latency.

Therefore, the results should not be interpreted as a deterministic end-to-end latency guarantee. In a deployment involving camera-stream transmission, network communication, or remote computation, additional sources of temporal variability would have to be incorporated into the system model and evaluation.

VI. DISCUSSION

The experimental results validate the core hypothesis: edge inference must be treated as a dynamic, hardware-aware process.

A. LIMITATIONS OF THE ACCURACY EVALUATION

The current accuracy results provide aggregate precision, recall, F1-score, and mAP@0.5 measurements for the evaluated 51-image set. However, they do not provide a separate small-object mAP measurement. Since potholes can occupy relatively small regions of the image, small-object-specific evaluation would provide a more direct characterization of the effect of resolution reduction.

Accordingly, the reported accuracy results should not be interpreted as a complete characterization of small-pothole detection performance. A dedicated small-object evaluation across the tested resolutions would be required to quantify this effect explicitly. The present results instead report the aggregate metrics available from the evaluated test set.

B. THE VALUE OF ANTICIPATION

The fundamental advantage of the learned controller over the deterministic heuristic is its ability to anticipate resource exhaustion. Thermal systems possess significant inertia: when T_{core} registers a high value, heat has already accumulated in the heatsink and temperatures will continue to rise even if workload is immediately reduced. The heuristic controller reacts only after this momentum is established. The learned policy, trained on transitions that capture the temporal relationship between current resource utilization and subsequent system state, learns to modulate $imgs_{sz_t}$ and $p_{skip,t}$ in a graded manner before approaching critical thresholds, achieving smoother throughput and lower peak temperatures.

C. INTERPRETATION OF BASELINE AND DETERMINISTIC CONTROL RESULTS

The baseline and deterministic-controller measurements justify the use of adaptive workload control at the application layer. The fixed 640×640 pipeline processed frames slowly and discarded most source frames because each frame required a substantial inference time. The deterministic controller reduced this mismatch by selecting lower-workload configurations when resource utilization increased, improving the measured throughput and reducing the frame-drop rate while keeping the observed core temperature below 60°C .

These results should not be interpreted as evidence that the controller eliminates all frame loss or guarantees hard real-time behavior. The remaining frame drops and repeated mode changes demonstrate the limitation of a stateless, threshold-based policy. This limitation provides the engineering justification for the later learned controller, whose objective is to select workload parameters using the longer-term consequences represented in the state-action-reward data.

D. INTERPRETATION OF OFFLINE CONTROLLER DEVELOPMENT

The Phase 4 and Phase 5 results establish the data and training stages of the learned-control pipeline, but they do not constitute an end-to-end deployment evaluation. The transition data provides coverage of continuous workload actions and their observed rewards, while reward filtering focuses training on actions associated with better measured operating outcomes. Because the available artifacts do not include a completed hardware-in-the-loop policy run, the learned controller should not yet be credited with the performance of the deterministic controller or with a measured accuracy improvement.

This distinction is important for interpreting the contribution: offline behavioral cloning removes exploratory policy updates from deployment, but it does not by itself prove that the learned policy generalizes to unseen thermal conditions. A subsequent evaluation should load the exported policy on the Raspberry Pi 5, replay the same workload, and report paired FPS, latency, temperature, frame-drop, and detection metrics against the static and deterministic baselines.

E. INTERPRETATION OF CROSS-PLATFORM MEASUREMENTS

The cross-platform plots are useful for checking whether the pipeline and its telemetry interfaces can be executed in different environments, but they are not a substitute for a controlled cross-platform experiment. Differences in processor, operating system, video decoding, thermal sensing, and runtime configuration can change the measured FPS and latency independently of the controller. This is why the manuscript treats the Raspberry Pi 5 runs as the primary evidence for the proposed edge deployment and reports other platforms as portability observations. A future comparison should use the same model, video, inference settings, run duration, telemetry definitions, and repeated trials on each target device.

F. INTERPRETATION OF ACCURACY ARTIFACTS

The accuracy-evaluation pipeline is a useful basis for the next experiment because it uses labeled images and an explicit IoU matching procedure. Its current outputs cannot be treated as final evidence, however, because the adaptive policy artifact is missing, the adaptive telemetry is simulated, and the saved reports are inconsistent. Reproducing the evaluation with one versioned policy, recorded hardware states, and a fixed output report is therefore necessary before making claims about mAP, F1, or precision changes. The same evaluation should report small-object performance separately and include a resolution sweep so that the safety cost of reduced input size is measured directly.

G. INTERPRETATION OF POLICY-ANALYSIS ARTIFACTS

The policy-analysis plots document the intended verification workflow, not a completed learning or deployment result. The verification script varies synthetic temperature, CPU, FPS, and latency values and then observes the actor's mapped actions. This can reveal malformed outputs or action-range violations, but it cannot demonstrate that the policy learned thermal dynamics, avoids unsafe actions, or improves run-time performance. Those properties must be tested with the exported policy, recorded Raspberry Pi 5 states, and paired baseline measurements before they are presented as results.

H. LOCAL ADAPTIVE INFERENCE VERSUS EDGE-CLOUD OFFLOADING

Edge-cloud task offloading provides an alternative mechanism for addressing computational limitations at resource-constrained devices. By distributing tasks between the local device, edge servers, and cloud infrastructure, offloading can provide access to additional computational resources while jointly considering communication and processing constraints [25]. This approach is particularly relevant for applications in which network connectivity and bounded communication latency can be reliably provided.

The present work instead adopts a connectivity-independent architecture in which both inference and control remain on the edge device. This design choice is motivated by

the deployment objective of maintaining local road-surface monitoring without requiring continuous communication with an external computing infrastructure. Offloading would introduce an additional communication path whose latency and availability depend on network conditions, in addition to the computational cost of transmitting video data or inference inputs. These factors are outside the scope of the current experimental evaluation.

This distinction does not imply that edge-cloud offloading is unsuitable for adaptive road-monitoring systems. Rather, the two approaches address different deployment assumptions. The proposed method reduces computational demand locally through adaptive resolution and frame-processing frequency, whereas an IoT-edge-cloud architecture can redistribute computation when suitable communication and remote computational resources are available. Integrating the proposed local controller with deterministic task offloading could therefore form a complementary hybrid architecture in future work, where local adaptation provides immediate workload control and offloading is invoked when network and remote-computing conditions satisfy the required service constraints.

I. SCOPE OF REPORTED LATENCY AND FPS METRICS

The FPS and latency figures reported in this work are local inference metrics: they measure the time to execute a single YOLOv8n forward pass on the Raspberry Pi 5, including pre-processing and post-processing. They do not represent end-to-end system latency from frame acquisition to any downstream consumer. The current architecture does not include a network transmission path; consequently, factors such as packet transmission delay, network jitter, queueing latency, or remote compute scheduling are not modelled and are not reflected in the reported metrics. If the system were extended to an edge-cloud deployment, these additional latency components would need to be incorporated into the evaluation.

J. OFFLINE EXPLORATION AND DEPLOYMENT POLICY

Exploratory actions are used exclusively during offline data collection (Phase 4). The data collection controller samples actions uniformly at random from $[-1, 1]^2$ with no reference to system safety or operating constraints. This exploration is performed in a bounded offline session (60 min / 20,000 transitions) on the target device. The trained policy deployed in Phase 5 is fully frozen: it performs only deterministic forward inference and does not perform online exploration, policy updates, or parameter modifications during operation. The deployed system therefore does not introduce any online learning-induced variability into the inference pipeline.

K. STATE DESIGN AND FUTURE CONTEXT-AWARE EXTENSIONS

The current state vector $s_t = [U_{cpu}, U_{ram}, T_{core}, FPS_{avg}, L_t]$ was designed to characterize the computational and thermal

operating condition of the hardware. It does not include scene-level variables such as vehicle speed, optical flow magnitude, or visual scene complexity. Including such variables could in principle enable context-aware adaptation: for example, reducing inference frequency when visual scene dynamics are low. This is a meaningful direction for future work but is not implemented in the current system. The current application scope is passive road-surface and infrastructure condition monitoring rather than safety-critical autonomous vehicle control; no safety guarantees from the adaptive controller are claimed or implied.

L. REWARD COEFFICIENT SELECTION AND SENSITIVITY

The reward coefficients (10, 0.5, 0.1, -50, -10) were selected to reflect the intended engineering hierarchy: detection quality as the primary objective, throughput as a secondary objective, latency penalized, and thermal/CPU overload conditions strongly discouraged. No formal hyperparameter optimization or sensitivity analysis of the reward coefficients was conducted. The selected values represent engineering design decisions. A systematic sensitivity analysis of reward coefficient choices and their effect on learned policy behavior is identified as future work.

M. GENERALIZATION TO OTHER MODELS

While this study utilized YOLOv8n, the Learned Adaptive Inference System is inherently model-agnostic. The defined action space modulates universal pipeline parameters—input resolution and frame-skip probability—rather than internal model weights. Consequently, the controller can be integrated with other detection or classification architectures, provided the action space is reprofiled for the target model's computational cost curve.

N. FUTURE RESEARCH DIRECTIONS

While the current implementation successfully demonstrates adaptive local inference, it opens several promising avenues for future exploration. First, augmenting the state vector with scene-level or visual-context features could enable content-aware adaptation, adjusting workloads dynamically based on visual scene dynamics. Second, conducting a systematic sensitivity analysis of the reward coefficients and investigating the data collection bounds across diverse operational environments present valuable opportunities to further refine policy robustness. Finally, while the current metrics focus on local inference, extending this framework into a hybrid edge-cloud architecture will allow future work to model and incorporate network transmission delays and end-to-end system latency.

VII. CONCLUSION

Deploying real-time object detection on resource-constrained edge devices requires moving beyond static inference pipelines that ignore hardware conditions during operation. This paper presented a Learned Adaptive Inference System that continuously adjusts image resolution and frame-

processing frequency based on real-time CPU utilization, RAM utilization, core temperature, FPS, and per-frame latency telemetry collected on a Raspberry Pi 5.

The system was developed in a structured six-phase pipeline. Phase 0 established an empirical Raspberry Pi 5 baseline showing that a fixed 640×640 YOLOv8n pipeline achieves a mean inference rate of 3.31 FPS and a cumulative frame-drop rate of 87.04% under sustained road-scene workloads. Phase 1 defined the action space as three characterization configurations for profiling resolution-workload trade-offs. Phases 2 and 3 introduced a deterministic heuristic controller using CPU and RAM thresholds, which improved throughput to 5.35 FPS and reduced the frame-drop rate to 41.91% while keeping core temperature below 60°C . This controller is interpretable and effective, but its stateless, reactive nature motivates a more capable learned alternative.

Phase 4 collected offline transition data by running a uniform random exploration policy on the target device, recording state-action-reward tuples over a bounded session. Phase 5 trained a compact Multi-Layer Perceptron via reward-filtered behavioral cloning, retaining only the top 30% of transitions by reward to bias learning toward high-performing operating configurations. Phase 6 exports the trained policy as a frozen TorchScript module that performs sub-millisecond deterministic inference on the Raspberry Pi 5 CPU without any online learning during deployment.

Together, these results demonstrate that adaptive local inference can meaningfully improve sustained throughput and reduce frame loss on resource-constrained hardware. The learned controller adds no training dependency to the deployment environment and introduces negligible inference overhead. Future work will evaluate the learned policy in a hardware-in-the-loop setting, augment the state vector with scene-level features, and explore hybrid edge-cloud architectures for environments with reliable network connectivity.

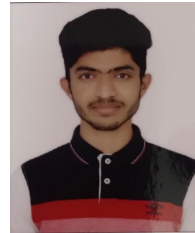
REFERENCES

- [1] R. Kothai and N. Prabhakaran, "Self Attention GAN and SWIN Transformer-Based Pothole Detection With Trust Region-Based LSM and Hough Line Transform for 2D to 3D Conversion," *IEEE Access*, vol. 13, pp. 137695–137708, Aug. 2025, doi: 10.1109/ACCESS.2025.3595190.
- [2] S. Kong, Q. Meng, X. Li, Z. Wang, X. Liu, and B. Li, "Road Perception for Autonomous Driving: Pothole Detection in Complex Environments Based on Improved YOLOv8," *IEEE Access*, vol. 13, pp. 80914–80929, May 2025, doi: 10.1109/ACCESS.2025.3566944.
- [3] G. Shelke et al., "Vision-Based Pothole Detection and Reporting System for Smart Cities (Leveraging YOLOv8-Seg and Predictive Maintenance Models)," in *Proc. IEEE Pune Section Int. Conf. (PuneCon)*, 2025, doi: 10.1109/PuneCon67554.2025.11379854.
- [4] K. Agarwal, S. Bhambri, V. K. Sridharan, N. Mohammed, H. Kohli, and J. A. Kapoor, "Performance Evaluation of different Machine Learning Techniques for Pothole Detection," in *Proc. IEEE Int. Conf. Contemporary Computing and Communications (InC4)*, 2025, doi: 10.1109/InC465408.2025.11256464.
- [5] M. Yurdakul and S. Tasdemir, "An Enhanced YOLOv8 Model for Real-Time and Accurate Pothole Detection and Measurement," arXiv preprint arXiv:2505.04207, May 2025.
- [6] Y. Hu et al., "Lightweight Deep Learning for Real-Time Road Distress Detection on Mobile Devices," *Nature Communications*, vol. 16, Art. no. 4212, May 2025.

- [7] N. Yang, S. Chen, H. Zhang, and R. A. Berry, "Beyond the Edge: An Advanced Exploration of Reinforcement Learning for Mobile Edge Computing, Its Applications, and Future Research Trajectories," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 1, pp. 546–594, 2025, doi: 10.1109/COMST.2024.3405075.
- [8] K. Ye et al., "Energy-Efficient Inference With Software–Hardware Co-Design for Sustainable Artificial Intelligence of Things," *IEEE Internet of Things Journal*, vol. 11, no. 24, pp. 39170–39182, Dec. 2024, doi: 10.1109/JIOT.2024.3482288.
- [9] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, "Dynamic Task Offloading in Edge Computing Based on Dependency-Aware Reinforcement Learning," *IEEE Transactions on Cloud Computing*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.
- [10] G. P. B. et al., "Accelerating Native Inference Model Performance in Edge Devices using TensorRT," in *Proc. IEEE Recent Advances in Intelligent Computational Systems (RAICS)*, 2024, doi: 10.1109/RAICS61201.2024.10690032.
- [11] "Enhancing Infrastructure Monitoring: Pothole Detection in Road Images Using YOLOv8 and Open Datasets," in *Proc. IEEE 6th Int. Conf. on Cybernetics and Intelligent System (ICORIS)*, 2024, doi: 10.1109/ICORIS63540.2024.10903697.
- [12] "Pothole Detection and Filling System Using Image Processing and Machine Learning," in *Proc. Int. Conf. Smart Systems for Applications in Electrical Sciences (ICSSES)*, 2024, doi: 10.1109/ICSSES62373.2024.10561417.
- [13] S. D. RS et al., "Pothole Detection and Instance Segmentation Using YOLO V8," in *Proc. IEEE Int. Conf. IoT Based Control Networks and Intelligent Systems (ICICNIS)*, 2024, doi: 10.1109/ICICNIS64247.2024.10823139.
- [14] R. C. C. M. Santos, M. Coelho, and R. Oliveira, "Real-Time Object Detection Performance Analysis Using YOLOv7 on Edge Devices," *IEEE Latin America Transactions*, vol. 22, no. 10, pp. 799–805, Oct. 2024, doi: 10.1109/TLA.2024.10705971.
- [15] R. M. et al., "An Annotated Street View Image Dataset for Automated Road Damage Detection (SVRDD)," *Scientific Data*, vol. 11, Art. no. 407, Apr. 2024.
- [16] A. R. Khouas et al., "Training Machine Learning Models at the Edge: A Survey," arXiv preprint, Jul. 2024.
- [17] "AdaDet: An Adaptive Object Detection System Based on Early-Exit Neural Networks," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 16, no. 1, pp. 332–345, Feb. 2024, doi: 10.1109/TCDS.2023.3274214.
- [18] X. Zhang, Y. Teng, N. Wang, B. Sun, and G. Hu, "Accelerating Deep Neural Network Tasks Through Edge-Device Adaptive Inference," in *Proc. IEEE 34th Annual Int. Symp. Personal, Indoor and Mobile Radio Communications (PIMRC)*, 2023, pp. 1–6, doi: 10.1109/PIMRC56721.2023.10293996.
- [19] S. Maity, R. Roy, A. Majumder, S. Dey, and A. R. Hota, "Future-Aware Dynamic Thermal Management in CPU-GPU Embedded Platforms," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2022, doi: 10.1109/RTSS55097.2022.00041.
- [20] G. Verma, Y. Gupta, A. M. Malik, and B. Chapman, "Performance Evaluation of Deep Learning Compilers for Edge Inference," in *Proc. IEEE Int. Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2021, pp. 858–865, doi: 10.1109/IPDPSW52791.2021.00128.
- [21] X. Gao, Y. Zhao, L. Dudziak, R. Mullins, and C. Xu, "Dynamic Channel Pruning: Feature Boosting and Suppression," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [22] B. Taylor, V. S. Marco, W. Wolff, Y. Elkhatib, and Z. Wang, "Adaptive Deep Learning Model Selection on Embedded Systems," in *Proc. 19th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environments*, 2018, pp. 31–43.
- [23] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," in *Proc. 23rd Int. Conf. Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.
- [24] M. Mutapic and S. Boyd, "Cutting-Set Methods for Robust Convex Optimization with Pessimizing Oracles," *Optimization Methods and Software*, vol. 24, no. 3, pp. 381–406, 2009.
- [25] K. Aghababaiyan, B. Coll-Perales, and J. Gozalvez, "Deterministic Task Offloading and Resource Allocation in the IoT-Edge-Cloud Continuum," in *Proc. IEEE 101st Vehicular Technology Conference (VTC2025-Spring)*, 2025, pp. 1–6, doi: 10.1109/VTC2025-Spring65109.2025.11174312.



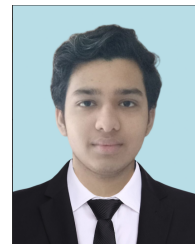
VISHAL K. BHAT is pursuing his Bachelor's degree in Computer Science at RV College of Engineering, Bengaluru, India. His research interests encompass computer vision, edge computing, and IoT systems. In this project, he led the development of the YOLOv8 detection pipeline and implemented the baseline profiling and runtime adaptive inference pipeline.



VAIBHAV P. RODAPPANAVAR is pursuing his Bachelor's degree in Computer Science at RV College of Engineering, Bengaluru, India. His research interests include machine learning, embedded systems, and adaptive inference algorithms. He contributed to the development of the reinforcement learning framework and the hardware abstraction layer for real-time system monitoring.



NAREN SRIDHARAN is pursuing his Bachelor's degree at RV College of Engineering, Bengaluru, India. His expertise includes secure edge deployments and system integration. He contributed to the experimental design, dataset management, and ensuring robust system recovery during thermal overload testing.



HARDIK PANERU is pursuing his Bachelor's degree at RV College of Engineering, Bengaluru, India. His research explores deep learning optimization and embedded software engineering. He formulated the Markov Decision Process (MDP) and contributed to the architecture utilized in the learned controller.



RAJESWARA RAO K V S is the Head of the Department of Industrial Engineering & Management at RV College of Engineering, Bengaluru, India. His research interests span industrial systems, process optimization, and the integration of artificial intelligence in engineering applications. He provided critical leadership, theoretical guidance, and project oversight for the adaptive edge inference system. ORCID: <https://orcid.org/0000-0003-1592-5578>.

...